Project #3: Managing Network for LLM Training

Technical Research Report: Predictive Models & Network Optimization

1. Introduction & Problem Statement

The rapid emergence of Large Language Models (LLMs) has necessitated distributed execution across massive GPU clusters. Conventional datacenter networks are often ill-equipped to handle the unique communication patterns of LLM training. This project addresses high operational costs, energy consumption, and the critical impact of GPU failures on training continuity.

2. Primary Challenges in LLM Networking

- **Network Bottlenecks:** Massive data exchange during gradient synchronization.
- **GPU Failures:** High-impact interruptions leading to resource wastage.
- **Traffic Congestion:** Inefficient routing leading to "bubbles" in the training pipeline.

3. Optimal Prediction & Routing Models

Research indicates that a multi-layered modeling approach is necessary to stabilize the network environment.

A. GPU Failure Prediction: Cascade Ensemble Model

The **Cascade Model-Ensemble** (utilizing Sliding Training Windows) is the current state-of-the-art. It improves failure prediction precision from roughly 46% to **85.4%** by combining Gradient Boosted Decision Trees (GBDT) with specialized classifiers to filter hardware telemetry noise.

B. Traffic Management: LCMP (Long-haul Cost-aware Multi-path)

For RDMA-heavy traffic, **LCMP** outperforms traditional ECMP routing. It utilizes a path-quality score to minimize tail latency (FCT slowdown) by up to **64%**.

4. Implementation Framework: ASTRA-sim 2.0

Component	Role in Project
Chakra Traces	Translates LLM execution logic (e.g., Llama-3) into network-readable communication patterns.
Topology Modeling	Simulates Fat-Tree or Spine-Leaf architectures to test routing efficiency.
Congestion Logic	Integrates predictive routing algorithms to handle "Elephant Flows" during All-Reduce operations.

5. Proposed Methodology

1. **Data Collection:** Use PyTorch Profiler and hardware telemetry to gather "normal" vs "failure" signatures.
2. **Model Training:** Implement a sliding window LSTM or Cascade Ensemble to identify pre-failure power/memory spikes.
3. **Simulation:** Inject failures into ASTRA-sim and measure the JCT (Job Completion Time) recovery using your optimized routing logic.